



Expert Recommendations — EdTech Island Platform

What I Would Improve If This Were My Codebase — Exhaustive Technical Breakdown

Scope: Architecture, Code Quality, Backend & DB, Security, UI/UX, New Features

Discipline: Ponytail Zero-Waste Engineering

Note: These are professional recommendations based on a full deep-read of your codebase, database schema, dev logs, and Obsidian architecture notes. Prioritized strictly by technical and security impact.

1. ARCHITECTURE

● Critical: Move to a Single Monorepo Build System

CRITICAL

Problem: You have 4 completely separate Vite builds (Teacher, Student, Admin, Landing) built with `npm run build:all` from root — but they share **zero code**. `supabase.js`, `ChatInterface.jsx`, `Card.jsx`, `ProfilePhotoModal.jsx`, `Layout.jsx` are all **copy-pasted 3–4 times** across portals.

Recommendation: Extract a `packages/shared/` workspace:

```
packages/  
  shared/  
    components/ ← Card, ChatInterface, Layout, ProfilePhotoModal  
    hooks/      ← usePresence, useAnnouncements, useTheme  
    lib/        ← supabase.js (single source of truth)  
portal-student/  
portal-teacher/  
portal-admin/
```

Use **npm workspaces** or **pnpm workspaces**. One change to `supabase.js` should propagate everywhere automatically.

● Critical: Study Island Should Be a React App, Not Vanilla JS

CRITICAL

Problem: `study-island/script.js` is **2,272 lines of vanilla JS** — a monolith. Adding new chapters requires editing this one massive file. It has no modularity, no component boundaries, no tree-shaking, and cannot be tested.

Recommendation: Convert Study Island to a Vite + React app. Each chapter becomes a lazy-loaded route:

```
/study-island/src/  
  chapters/  
    science/LightAndShadows/ ← lazy loaded  
    science/ReflectionMirrors/  
    math/SpatialGeometry/  
  components/  
    ChapterShell.jsx          ← the 5-tab wrapper  
    ExperimentCarousel.jsx  
    QuizRunner.jsx
```

New chapters = new folders. Zero risk to existing chapters.

● Important: Hardcoded Port Origins & Student Context

IMPORTANT

Problem: You fixed `http://localhost:3000` in some places but I still see `classList = ['Class 1st' ...]` style arrays and `mockClasses` that assume the environment is always the same. The bigger architectural gap: **Study Island has no way to know what class/student is using it** — there's no user session passed to it.

Recommendation: When launching Study Island, pass an encrypted JWT or Supabase session token in the URL and let Study Island read it to know which student is viewing which chapter. This enables per-student progress tracking inside the 3D app.

● Important: No API Layer — All Supabase Queries Are in Components

IMPORTANT

Problem: Every component directly calls `supabase.from('table').select(...)`. Business logic is tangled with UI. If you ever change a table name or query structure, you have to hunt through 30+ JSX files.

Recommendation: Create a thin `services/` layer:

```
src/services/  
classService.js  ← all queries for classes table  
testService.js   ← all queries for tests table  
userService.js   ← auth + profile queries
```

Components call `classService.getAll()` — they never touch Supabase directly.

2. CODE QUALITY

● Critical: Progress.jsx Is a Static Mock — Not Real Data

CRITICAL

Problem: The entire academic progress page for students is fake. `chartDataByRange` is a hardcoded object with static numbers. The study streak is hardcoded "10 Days". Subject mastery percentages are hardcoded. **Students are looking at fake data and don't know it.**

Fix: Connect to `quiz_scores` and `student_test_attempts` tables. Calculate: Actual score history per date, actual subject-wise average from quiz results, and real streak from `last_active_date` in profiles.

● Critical: Test Scores Stored in localStorage Only

CRITICAL

Problem: When a student takes a quiz in `TestRunnerModal`, the result is saved to `localStorage['student_test_results']`. If the student clears browser data, uses a different device, or opens an incognito window — **all their results are gone.**

Fix: After grading, INSERT the result into a `test_submissions` table:

```
test_submissions (
  id, student_id, test_id, score, grade,
  answers_json, submitted_at, attempt_number
)
```

Keep `localStorage` only as a cache.

● Important: ToDoList.jsx Is Ephemeral & Hardcoded Class Dropdowns

IMPORTANT

Problem: `ToDoList.jsx` has 0% backend integration and resets on refresh. Class dropdowns (`mockClasses = ['Class 1st' ...]`) are copy-pasted across 5+ files.

Fix: Wire `ToDo` list to Supabase `todos` table. Replace static class arrays with a single cached `useClasses()` hook.

● Minor: Loading Skeletons & Console Log Cleaning

MINOR

Components show raw white flashes while loading data. Console logs are visible in production builds.

Add shimmer CSS skeleton components during fetch states. Strip console logs in `vite.config.js` using `esbuild.drop: ['console', 'debugger']`.

3. BACKEND & DATABASE

● Critical: No Row-Level Security (RLS) Enforced Anywhere

CRITICAL

Problem: All Supabase queries use the public anon key. Any user can open DevTools and read **every student's test scores, teachers' personal data, and private chat messages**. RLS is mandatory before storing real student data.

Minimum RLS Policies Needed:

```
-- Students can only read their own scores
CREATE POLICY "students_own_scores" ON quiz_scores
  USING (auth.uid() = student_id);

-- Messages only visible to conversation participants
CREATE POLICY "message_visibility" ON messages
  USING (
    auth.uid() IN (
      SELECT user_id FROM conversation_participants
      WHERE conversation_id = messages.conversation_id
    )
  );
```

```
)  
);
```

● Important: Missing student_class_enrollment & Schema Inconsistencies

IMPORTANT

Problem: No table links students to classes, causing Class 6 students to see tests for Class 8. announcements queries mix createdAt (camelCase) and created_at (snake_case). Database updates lack migration scripts.

Create student_enrollments table. Standardize PostgreSQL columns on created_at. Use **Supabase CLI** for version-controlled migration files.

4. SECURITY

● Critical: Auth is Entirely Client-Side Trust

CRITICAL

Problem: Role checks (`if user.role === 'teacher'`) happen in React state. Students can alter `localStorage.edtech_teacher_user` to access teacher tools.

Fix: Every sensitive query and action must be verified server-side using Supabase `auth.uid()` in RLS policies. Never trust client state.

● Critical: URL Injection in Student Chat (`Chats.jsx`)

CRITICAL

Problem: `Chats.jsx` reads `?user=` from URL and writes it to `localStorage.setItem('edtech_user', userParam)`. This is a stored XSS session hijacking vector.

Fix: Remove URL parameter user parsing. Session identity must come exclusively from Supabase Auth tokens.

● Important: Rate Limiting on Upload Endpoints

IMPORTANT

`/api/upload-r2` in `server.js` has no rate limiting. Malicious scripts could spam file uploads to R2 and incur storage costs.

Add IP-based rate limiting (max 10 uploads per minute per IP).

5. UI / UX OPTIMIZATIONS

● Critical: Mobile Responsiveness & Poor Network Handling

CRITICAL

Portals are desktop-only (fixed 240px sidebars). Mobile browser experience is broken. Offline network states show blank white screens.

Add CSS breakpoint media queries (collapsible mobile hamburger drawer). Add offline connection banners and fallback `sessionStorage` caching.

● Important: Empty States & Accessibility (WCAG 2.1)

IMPORTANT

Empty lists show blank whitespace. Icon buttons lack `aria-label` and keyboard focus rings. Destructive actions lack typed confirmations.

Add friendly empty state cards with CTA buttons (`"No tests created yet — [+ Create Test]"`). Add WCAG compliant ARIA labels and styled confirmation modals.

6. TOP RECOMMENDED MISSING FEATURES

1. Student Attendance Tracking System

FEATURE

No attendance system exists. Teachers cannot mark present/absent students.

Build: Teacher class attendance grid → Supabase attendance table → Admin monthly attendance reports → Parent alerts.

2. Study Island Gamification & XP System

FEATURE

Study Island 3D is missing a core game loop to drive student retention.

Build: Earn XP for completing 3D chapters (+50 XP), 90%+ quiz scores (+100 XP), and 7-day study streaks. Render class leaderboards in Student & Teacher views.

3. AI-Generated Quiz from Chapter Content (Gemini API)

FEATURE

Teachers must manually type every single question in the Question Bank.

Build: Upload chapter PDF/notes → Gemini API scans content → auto-generates 10 MCQs → teacher approves with 1-click.

4. Web Push Notification Engine & Parent Portal

FEATURE

No push notifications when live classes start. No dedicated view for parents.

Build: Native Web Push API alerts for live class starts and test assignments. Build read-only Parent Portal for student attendance & grades.

SUMMARY PRIORITY ACTION MATRIX

PRIORITY	CATEGORY	ITEM / FOCUS AREA	EXPECTED OUTCOME
P0 Urgent	Security	Supabase Row-Level Security (RLS)	Blocks unauthorized database reads/writes in DevTools
P0 Urgent	Security	Remove URL ?user= parsing	Eliminates stored XSS and role spoofing vulnerabilities
P0 Urgent	Data Integrity	Real Progress engine & Test submissions	Stores quiz attempts in DB & renders real score graphs
P1 High	Architecture	Shared packages/shared/ Monorepo	Single source of truth for Supabase client & shared UI
P1 High	UI / UX	Mobile Responsive Layouts	Enables mobile usage for students across India
P1 High	Feature	Student Attendance System	Completes essential school administrative suite
P2 Medium	Feature	Study Island Gamification & XP	Drives daily student retention and chapter completion
P2 Medium	Feature	AI Quiz Generation (Gemini API)	Automates question generation for teachers